COLLECTION TOTAL — RÉCONCILIATION

Livre 2 sur 21

Le SDLC Réconcilié

Maîtriser le cycle de vie logiciel de bout en bout

Partie A sur 3 · SENSE — Faire sens, comprendre, définir

Une publication de la collection **ToTAL — Réconciliation**

Domaines : **SPECIFY** (cs) · **DELIVERY** (ci/cd)

Zones : **SENSE** · **FORGE** · **CARE**

Phases : ÉLICITER · DÉFINIR · PLANIFIER · IMPLÉMENTER · DIFFUSER · CONFIRMER

Crédits visuels : images et couverture par **Soléane Bélanger**

Table des matières

Avant-propos — Pourquoi un livre entier sur le SDLC ?

Repères de lecture

Comment utiliser ce livre

Conventions de lecture

Introduction — Le cycle qui réconcilie tout

Pourquoi les équipes perdent la maîtrise du cycle

Le SDLC réconcilié : 6 phases, 3 zones, 2 domaines, grande boucle

Vue synthèse : tableau des 6 phases

La grande boucle : un ruban de Möbius

Partie 1 — SENSE : Faire sens et comprendre

Chapitre 1 — ÉLICITER : recueillir les signaux et comprendre les besoins

Ce que ÉLICITER n'est pas

Les 5 objets d'élicitation

Techniques d'élicitation ToTAL

La DoR ToTAL

Scène ORIGINALIUM : la douleur cachée

Carnet de terrain

Chapitre 2 — ÉLICITER en pratique : les 5 profils de sources de besoin

Les 5 profils détaillés

Grille : source → signal → question → risque

Prioriser les sources selon le contexte

Chapitre 3 — DÉFINIR : transformer le besoin en spécification observable

La spécification ToTAL

La forme canonique SI/ALORS

Une spécification atomique

Le piège des spécifications décoratives

La DoD-spec ToTAL

Scène ORIGINALIUM : Karim demande « comment je teste ça ? »

Chapitre 4 — DÉFINIR en pratique : niveaux et types de spécification

Niveaux de granularité

Types de spécification

La traçabilité ToTAL

Spécification et testabilité

Carnet de terrain : le vertige de dire

Chapitre 5 — SENSE en pratique : conduire l'atelier ÉLICITER-DÉFINIR

L'atelier SENSE ToTAL

Outils de l'atelier

Rôles en atelier SENSE

Avant-propos — Pourquoi un livre entier sur le SDLC ?

La première fois qu'on lit la définition du SDLC — cycle de développement logiciel, six phases, deux domaines, trois zones — on peut avoir l'impression de tenir quelque chose de simple. Une carte. Une boussole. Une façon élégante de nommer ce que les équipes font depuis des décennies. Et c'est vrai : dans le Livre 1 de cette collection, le SDLC réconcilié est présenté comme une carte. Une vue d'ensemble. Un cadre de navigation.

Mais une carte, aussi précise soit-elle, n'apprend pas à marcher. Elle ne dit pas comment sentir le terrain sous ses pieds. Elle ne prévient pas des ornières invisibles, des raccourcis trompeurs, des moments où l'équipe croit avancer mais tourne en rond. C'est là qu'un livre entier sur le SDLC devient non seulement utile — il devient nécessaire.

Dans ce Livre 2, le SDLC réconcilié cesse d'être une carte et devient une pratique. Chaque phase est disséquée avec la profondeur qui lui manquait dans L1. Les entrées et les sorties sont nommées. Les pièges sont exposés. Les transitions entre phases sont expliquées non comme des règles administratives, mais comme des moments de décision qui changent la trajectoire de tout un projet.

La promesse de ce livre

À la fin de ce Livre 2 — à travers ses trois fascicules, Partie A, Partie B et Partie C — le lecteur sera en mesure de conduire une équipe à travers les 6 phases du SDLC réconcilié avec une maîtrise réelle. Pas théorique. Pas simulée. Réelle : avec les bonnes questions à poser à chaque phase, les critères concrets pour savoir qu'on peut

passer à la suivante, et les signaux d'alarme qui indiquent qu'on ne devrait pas le faire.

Cette Partie A couvre la première zone : **SENSE**. Faire sens. Comprendre. Définir. C'est le territoire des deux premières phases du cycle — ÉLICITER et DÉFINIR — et c'est sans doute le territoire le plus sous-estimé, le plus bâclé, et le plus coûteux à négliger de tout le cycle logiciel.

Ce que ce livre n'est pas

Ce livre n'est pas un retour à la cascade. Le SDLC réconcilié ne propose pas de tout spécifier avant de toucher au code, puis de tout construire avant de montrer quoi que ce soit. Cette époque est révolue, et les cicatrices laissées par des décennies de projets livrés trop tard, trop cher, et trop loin des besoins réels en sont la preuve.

Mais ce livre n'est pas non plus de l'Agile déguisé. Il ne prétend pas qu'en découpant le travail en sprints de deux semaines et en tenant des stand-ups quotidiens, les équipes vont naturellement converger vers les bons résultats. L'agilité sans structure produit de la vélocité sans direction — et la vélocité sans direction est une autre façon de se perdre, juste plus rapidement.

Le SDLC réconcilié est une troisième voie. C'est une trajectoire de maîtrise qui réconcilie les approches : la rigueur de la pensée structurée avec la souplesse de l'apprentissage continu. La précision des critères de sortie avec l'honnêteté des incertitudes persistantes. La discipline du SPECIFY avec la fluidité du DELIVERY. Ce n'est pas un compromis entre deux mondes. C'est une synthèse qui en crée un troisième.

Un mot sur BEASTEST

Tout au long de ce livre, vous rencontrerez des références à **BEASTEST**. BEASTEST est un outil optionnel — il peut faciliter et accélérer certaines pratiques décrites dans ces pages, notamment dans les phases d'élicitation et de définition. Mais aucune des idées, des techniques ou des frameworks présentés ici ne dépend de BEASTEST. Tout ce que vous lirez peut être mis en pratique avec les outils que vous avez déjà : un

tableau blanc, un carnet, une équipe engagée, et la volonté de travailler avec méthode.

Repères de lecture

Comment utiliser ce livre

Ce fascicule a été conçu pour deux types de lecteurs, et il s'adapte aux deux.

Le **lecteur linéaire** — celui qui lit de la première à la dernière page — trouvera un récit cohérent : les chapitres se construisent les uns sur les autres, et chaque notion introduite devient le fondement de la suivante. Si vous découvrez le SDLC réconcilié pour la première fois, ou si vous souhaitez en avoir une compréhension complète et articulée, la lecture linéaire est recommandée.

Le **lecteur par phase** — celui qui revient sur un chapitre spécifique parce qu'il en a besoin maintenant, dans le contexte d'un projet en cours — trouvera que chaque chapitre est suffisamment autonome pour être consulté seul. Les boîtes récapitulatives, les tableaux de synthèse et les phrases repères permettent de naviguer rapidement vers l'essentiel.

Dans tous les cas, les éléments suivants sont présents de façon récurrente dans ce livre :

- **Scènes ORIGINALIUM** : des récits narratifs mettant en scène l'équipe DREAM-TEAM — Rania (tech lead), Théo (PO) et Karim (QA) — dans des situations concrètes inspirées de la réalité du terrain logiciel.
- **Carnets de terrain** : des réflexions plus personnelles, tirées d'expériences vécues, qui ancrent les concepts dans la texture rugueuse des vrais projets.
- **Boîtes thématiques** : À retenir, Symptômes à observer, Questions de réflexion, Signal de vigilance.
- **Tableaux de synthèse** : pour les grilles, les critères, les comparaisons.

- **Phrases repères** : des formulations courtes qui condensent une idée centrale et peuvent être affichées dans l'espace de travail d'une équipe.

Conventions de lecture

Ce livre utilise un ensemble stable de conventions qu'il est utile de connaître avant de commencer.

Convention	Signification	Exemple
SPECIFY (cs)	Domaine de la spécification — comprendre, définir, rendre observable	ÉLICITER et DÉFINIR appartiennent au domaine SPECIFY
DELIVERY (ci/cd)	Domaine de la livraison — construire, diffuser, confirmer	IMPLÉMENTER et DIFFUSER appartiennent au domaine DELIVERY
SENSE	Zone regroupant ÉLICITER + DÉFINIR	Faire sens, comprendre, préparer la décision
FORGE	Zone regroupant PLANIFIER + IMPLÉMENTER	Construire, réaliser, livrer techniquement
CARE	Zone regroupant DIFFUSER + CONFIRMER	Déployer, observer, apprendre
SI/ALORS	Forme canonique d'une spécification ToTAL	SI [condition observable] ALORS [résultat observable attendu]
DoR	Definition of Ready — critère de sortie d'ÉLICITER	Un besoin est prêt à être défini
DoD	Definition of Done — critère de sortie d'une phase ou d'un incrément	Un incrément est terminé selon les critères convenus
DoT	Definition of Tested — preuve que le comportement a été vérifié	Un test automatisé ou manuel confirme le résultat attendu
Grande boucle	Retour de CONFIRMER vers ÉLICITER — ruban de Möbius	L'apprentissage d'une livraison alimente le prochain cycle

> 📖 **Note sur la langue**
>
> Ce livre est rédigé en français québécois professionnel. Certains termes anglais sont conservés lorsqu'ils sont d'usage universel dans l'industrie logicielle (SDLC, DoR, DoD, backlog, sprint, pipeline). Ces termes sont toujours présentés avec leur équivalent fonctionnel en français lors de leur première apparition.

Introduction — Le cycle qui réconcilie tout

Pourquoi les équipes perdent la maîtrise du cycle

Posez la question à n'importe quelle équipe logicielle : « Connaissez-vous votre cycle de développement ? » La réponse sera presque toujours oui. Et elle sera sincère. Les équipes savent qu'elles ont un cycle. Elles connaissent les phases, au moins de nom. Elles ont des réunions, des rituels, des outils.

Mais posez la question autrement : « Savez-vous précisément à quelle phase vous êtes aujourd'hui, et sur quels critères vous déciderez de passer à la suivante ? » Le silence qui suit est révélateur.

La perte de maîtrise du cycle logiciel ne survient pas d'un coup. Elle s'installe progressivement, par accumulation de petites décisions non prises, de transitions non formalisées, de critères non énoncés. Elle commence souvent dans les premières phases — celles qu'on bâcle parce qu'on veut commencer à construire. Et elle se révèle toujours trop tard : en tests, en production, dans la bouche d'un utilisateur qui dit « c'est pas ce qu'on voulait ».

Il existe au moins trois grandes façons dont les équipes perdent la maîtrise :

- **La confusion des phases :** ÉLICITER et DÉFINIR sont fusionnées en une seule « réunion de requirements ». PLANIFIER est sautée au profit d'un « on commence et on voit ». CONFIRMER est réduite à un test de fumée avant déploiement.
- **L'absence de critères de sortie :** On passe d'une phase à l'autre par épuisement du temps alloué, non par satisfaction de critères convenus. Le travail « terminé » est en réalité un travail « abandonné ».
- **La rupture de la grande boucle :** Ce qu'on apprend d'une livraison n'est jamais réinjecté dans le prochain cycle d'élicitation. Chaque projet repart de zéro dans ses erreurs.

Le SDLC réconcilié est une réponse directe à ces trois échecs. Pas une réponse théorique — une réponse pratique, outillée, vérifiable.

Le SDLC réconcilié : 6 phases, 3 zones, 2 domaines, grande boucle

Le SDLC réconcilié est une structure en six phases, organisées en trois zones, elles-mêmes réparties dans deux domaines, le tout traversé par une grande boucle d'apprentissage continu.

> SENSE [ÉLICITER · DÉFINIR] → FORGE [PLANIFIER · IMPLÉMENTER] → CARE [DIFFUSER · CONFIRMER] ↩️ ÉLICITER

Chaque élément de cette structure a une raison d'être précise :

- **Les 6 phases** (ÉLICITER, DÉFINIR, PLANIFIER, IMPLÉMENTER, DIFFUSER, CONFIRMER) nomment chaque moment de transformation du cycle : du signal brut de besoin jusqu'à la preuve d'apprentissage.

- **Les 3 zones** (SENSE, FORGE, CARE) regroupent les phases par nature de travail et permettent de raisonner à un niveau intermédiaire — ni trop fin (phase par phase), ni trop haut (l'ensemble du cycle).
- **Les 2 domaines** (SPECIFY et DELIVERY) tracent la grande ligne de partage entre penser et faire, entre rendre observable et rendre réel.
- **La grande boucle** (CONFIRMER → ÉLICITER) est le mécanisme d'apprentissage organisationnel. Ce n'est pas un retour en arrière — c'est une progression en spirale, où chaque retour enrichit le prochain départ.

Vue synthèse : tableau des 6 phases

Zone	Phase	Domaine	Entrée principale	Sortie principale	Critère de sortie
SENSE	**ÉLICITER**	SPECIFY	Signal brut (douleur, demande, observation)	Besoin compris et nommé	DoR — Definition of Ready
DÉFINIR	SPECIFY	Besoin compris et nommé	Spécification observable et testable (SI/ALORS)	DoD-spec — Definition of Done spécification	
FORGE	**PLANIFIER**	DELIVERY	Spécification validée	Plan d'incrément, architecture, risques identifiés	DoP — Definition of Planned
IMPLÉMENTER	DELIVERY	Plan validé, spécifications prêtes	Incrément fonctionnel livrable	DoD — Definition of Done	
CARE	**DIFFUSER**	DELIVERY	Incrément validé	Déploiement en environnement cible	DoR-déploiement — prêt à livrer
CONFIRMER	DELIVERY	Déploiement effectué	Apprentissage validé, signal de retour	DoT — Definition of Tested + signal d'apprentissage	

La grande boucle : un ruban de Möbius — toujours en avant

La grande boucle du SDLC réconcilié — celle qui relie CONFIRMER à ÉLICITER — est souvent mal interprétée. On pense « retour en arrière ». On pense « boucle fermée ». On pense que recommencer depuis ÉLICITER signifie tout recommencer depuis zéro.

C'est une erreur de lecture profonde.

La grande boucle est un ruban de Möbius : une surface continue qui, en apparence, revient à son point de départ, mais sur une face différente. Ce qui revient à ÉLICITER après CONFIRMER n'est pas la même équipe, avec le même niveau d'ignorance, devant le même problème vierge. C'est une équipe enrichie par l'apprentissage d'un cycle complet, qui aborde le prochain besoin avec une compréhension plus fine des vrais problèmes, des vraies contraintes, et des vraies valeurs de ses utilisateurs.

> *La grande boucle n'est pas un retour en arrière. C'est la façon dont le système apprend à mieux voir ce qu'il n'avait pas encore vu. Chaque tour du cycle est une révolution — au sens propre du mot : une rotation complète qui laisse l'observateur à un niveau plus élevé qu'au départ.*

Cette distinction n'est pas philosophique. Elle est opérationnelle. Une équipe qui comprend la grande boucle comme un mécanisme d'apprentissage va structurer sa phase CONFIRMER différemment — non pas pour « cocher une case de validation », mais pour extraire des signaux précis qui vont directement nourrir le prochain cycle d'élicitation. Le signal qui sort de CONFIRMER est l'entrée de la prochaine itération d'ÉLICITER. La boucle n'est pas une fin — c'est un départ amélioré.

> 💡 **À retenir**

Le SDLC réconcilié n'est pas un retour à la cascade. Ce n'est pas de l'Agile déguisé. C'est une structure de maîtrise : 6 phases nommées, 3 zones articulées, 2 domaines distincts, et une grande boucle d'apprentissage continu qui avance toujours — jamais en arrière, toujours en avant.

Partie 1 — SENSE : Faire sens et comprendre

Zone SENSE

SENSE est la première zone du SDLC réconcilié. Elle regroupe les deux premières phases du cycle — ÉLICITER et DÉFINIR — et couvre l'ensemble du territoire qu'on pourrait appeler « avant de construire » : comprendre ce qui mérite d'être construit, et le formuler de façon à ce que sa construction puisse être vérifiée.

C'est la zone la plus sous-investie et la plus surchargée de toutes. Sous-investie parce qu'on la bâcle pour « aller construire ». Surchargée parce que quand elle est mal faite, toutes les autres phases doivent compenser ses lacunes — en tests de régression, en corrections de dernière minute, en délais accumulés, en livrables qui manquent leur cible.

SENSE ne prépare pas seulement ce qu'on va construire. Elle prépare la confiance dans ce qu'on va décider.

Chapitre 1 — ÉLICITER : recueillir les signaux et comprendre les besoins

SENSE · Phase 1

Si vous deviez choisir une seule phase du SDLC réconcilié pour y investir plus de temps, plus d'attention, plus de soin — ce serait ÉLICITER. Non pas parce qu'elle est plus noble que les autres, mais parce qu'elle est la fondation sur laquelle tout le reste repose. Et les fondations mal coulées ne se révèlent jamais lors du coulage : elles se révèlent quand l'édifice commence à fléchir, des mois plus tard, sous le poids de la réalité.

ÉLICITER est la phase de recueil. Recueil des signaux, des besoins, des douleurs, des espoirs, des contraintes, des contextes. C'est le moment où une équipe sort de sa zone de confort technique pour entrer dans le territoire humain — le territoire des personnes qui ont un problème, qui attendent une valeur, qui subissent une situation, qui anticipent un risque.

Et c'est pour ça qu'ÉLICITER est si difficile. Pas techniquement. Humainement.

Ce que ÉLICITER n'est pas

Avant de définir ce qu'est ÉLICITER, il est utile de clarifier ce qu'elle n'est pas — parce que les confusions ici sont nombreuses, répandues, et coûteuses.

ÉLICITER n'est pas une réunion de collecte de requirements. La réunion de requirements est un artefact d'une époque révolue où on croyait que les besoins existaient quelque part, déjà formés, en attente d'être recueillis comme des fruits mûrs sur un arbre. Il suffisait de poser des questions, de noter les réponses, et de repartir avec une liste. Cette vision est fausse. Les besoins ne sont pas des données statiques — ce sont des constructions sociales, contextuelles, évolutives. Ils changent quand on les nomme. Ils se transforment quand on les discute. Ils révèlent souvent quelque chose d'inattendu quand on les observe plutôt que de les demander.

ÉLICITER n'est pas une phase de documentation. Ce n'est pas le moment d'écrire des pages de spécification. C'est le moment de comprendre — ce qui est fondamentalement différent. L'écriture vient après. L'écriture prématurée tue l'écoute.

ÉLICITER n'est pas un travail du seul Product Owner. Oui, le PO joue un rôle central. Mais limiter l'élicitation au PO, c'est priver l'équipe d'une richesse de perspectives qui ne se révèle que quand plusieurs types d'intelligence sont dans la même pièce. Le regard du tech lead sur la faisabilité, l'œil du QA sur la vérifiabilité — ces perspectives enrichissent l'élicitation dès qu'elles y participent, pas après.

Les 5 objets d'élicitation

Dans le SDLC réconcilié, l'élicitation s'intéresse à cinq types d'objets. Ces cinq objets ne sont pas des catégories rigides — ils se chevauchent, se nourrissent mutuellement, et l'art de l'élicitation consiste en partie à naviguer entre eux avec fluidité.

#	Objet	Question centrale	Ce qu'on cherche à comprendre
1	**La douleur**	Qu'est-ce qui fait souffrir aujourd'hui ?	Le problème vécu, ressenti, subi — souvent caché derrière une demande de solution
2	**La valeur**	Qu'est-ce qui a de l'importance si ça change ?	Ce que la personne ou l'organisation cherche à obtenir, protéger ou amplifier
3	**Le risque**	Qu'est-ce qui pourrait mal tourner ?	Les incertitudes, les fragilités, les contraintes non exprimées, les hypothèses implicites
4	**La décision**	Qui décide, sur quoi, et avec quelles	Le circuit de décision réel (souvent différent

#	Objet	Question centrale	Ce qu'on cherche à comprendre
		informations ?	de l'organigramme officiel)
5	**Le signal**	Qu'est-ce qui indique que quelque chose change ou a changé ?	Les indicateurs observables qui permettront de vérifier que la valeur a été créée

Ces cinq objets forment ensemble un portrait complet du territoire à explorer. Une élicitation qui ne couvre que la douleur et la valeur, mais ignore le risque, la décision et le signal, produira des besoins compris à moitié — et des spécifications testables à moitié.

Techniques d'élicitation ToTAL

Le SDLC réconcilié ne prescrit pas une technique unique d'élicitation. Il reconnaît que les contextes varient — un projet interne B2B n'est pas une startup produit grand public, et un système embarqué critique n'est pas une application SaaS de gestion. Ce qui reste constant, c'est la discipline : choisir la technique en fonction du contexte, l'appliquer avec rigueur, et en extraire des signaux exploitables.

L'entretien structuré

L'entretien structuré est la technique la plus fondamentale. Une conversation individuelle ou en petit groupe, guidée par un cadre de questions ouvertes, organisée autour des 5 objets d'élicitation. L'entretien structuré n'est pas un questionnaire — c'est une conversation avec une colonne vertébrale. L'intervieweur sait où il veut aller, mais il sait aussi s'arrêter quand quelque chose d'inattendu émerge.

La règle d'or de l'entretien structuré : *la question la plus précieuse n'est pas celle que vous aviez planifiée — c'est celle que vous poserez en réaction à ce que vous venez d'entendre.*

L'observation directe

Observer l'utilisateur dans son contexte réel est souvent plus révélateur que de lui demander ce dont il a besoin. Les gens font rarement ce qu'ils disent faire — non par mauvaise foi, mais parce que leur perception de leur propre comportement est filtrée par des habitudes invisibles, des ajustements inconscients, des contournements intégrés.

L'observation directe révèle les douleurs que personne ne pensait à mentionner parce qu'elles semblaient normales. Elle révèle aussi les usages non prévus des systèmes existants — les « hacks » que les utilisateurs ont développés pour compenser les lacunes du logiciel, et qui sont autant de besoins non articulés en attente d'être entendus.

La carte d'empathie

La carte d'empathie est un outil collaboratif qui permet à une équipe de construire collectivement une représentation de l'expérience d'un utilisateur. Elle s'organise autour de six dimensions : ce que l'utilisateur dit, pense, ressent, fait, entend, et voit. En remplissant cette carte ensemble, les membres d'une équipe apprennent non seulement ce que l'utilisateur vit — ils apprennent aussi où leurs propres perceptions divergent, ce qui est en soi une information précieuse sur leurs angles morts collectifs.

L'atelier de signal

L'atelier de signal est une technique propre au SDLC réconcilié. Il s'agit d'un atelier court — typiquement 60 à 90 minutes — où l'équipe et les parties prenantes travaillent ensemble sur une question précise : « Qu'est-ce qui devrait changer pour que nous sachions que nous avons créé de la valeur ? »

L'atelier de signal force la conversation vers le concret observable. Il est particulièrement efficace pour identifier les signaux qui alimenteront la phase CONFIRMER — et il crée très tôt dans le cycle un alignement sur ce que « succès » signifie concrètement pour tout le monde.

💡 **À retenir — ÉLICITER**

ÉLICITER n'est pas une réunion de collecte de requirements. C'est une exploration disciplinée du territoire humain autour d'un besoin. Elle s'intéresse à 5 objets (douleur, valeur, risque, décision, signal) et mobilise 4 techniques principales (entretien structuré, observation directe, carte d'empathie, atelier de signal). Son résultat n'est pas un document — c'est une compréhension partagée suffisamment solide pour entrer dans DÉFINIR.

Critère de sortie d'ÉLICITER : la Definition of Ready (DoR)

La phase ÉLICITER se termine lorsqu'un besoin est suffisamment compris pour entrer dans la phase DÉFINIR. Ce moment est formalisé par la **Definition of Ready (DoR)** — la définition de « prêt à être défini ».

La DoR n'est pas une déclaration de certitude totale. Elle est une déclaration de suffisance : le besoin est assez compris pour qu'on puisse maintenant travailler à le rendre spécifiable. Il y a toujours des zones d'ombre. La DoR dit : « les zones d'ombre restantes ne nous empêchent pas de définir — elles seront adressées dans des itérations suivantes. »

La DoR ToTAL : 5 conditions

#	Condition	Question de validation	Signe que la condition n'est pas remplie
1	**La douleur est nommée**	Peut-on décrire la situation problématique actuelle en une phrase claire, sans mentionner de solution ?	On décrit la solution demandée, pas le problème vécu
2	**La valeur est articulée**	Sait-on ce que la résolution de cette	La valeur est vague, générique

#	Condition	Question de validation	Signe que la condition n'est pas remplie
		douleur change concrètement pour qui ?	(« améliorer l'expérience utilisateur »)
3	**Le risque principal est identifié**	A-t-on nommé au moins une hypothèse critique dont la fausseté changerait radicalement notre approche ?	Aucune hypothèse n'a été explicitée — tout semble certain et simple
4	**La décision est claire**	Sait-on qui a le pouvoir de valider que ce besoin mérite d'être traité, et comment cette décision sera prise ?	Flou sur qui peut dire oui, ou sur les critères de priorisation
5	**Un signal de confirmation est esquissé**	A-t-on une idée — même grossière — de ce qui sera différemment observable si le besoin est adressé ?	On n'a aucune idée de comment on saura que ça a fonctionné

⚠ **Signal de vigilance — DoR non atteinte**

Un besoin qui entre en phase DÉFINIR sans DoR validée produit presque toujours une spécification boiteuse. Une spécification boiteuse produit des tests tardifs et fragiles. Des tests tardifs et fragiles produisent des corrections coûteuses en phase IMPLÉMENTER ou DIFFUSER. La pression pour « avancer vite » en sautant la DoR est l'une des sources les plus fréquentes de dette de qualité accumulée.

Signaux d'alarme : quand ÉLICITER a été mal faite

Il est utile de savoir reconnaître, à des stades avancés du cycle, les traces d'une élicitation insuffisante. Ces signaux apparaissent généralement en phase

IMPLÉMENTER, DIFFUSER ou CONFIRMER — et leur coût de correction à ce stade est bien supérieur à ce qu'il aurait été si on avait investi plus tôt dans ÉLICITER.

🔍 **Symptômes à observer**

- Les développeurs reçoivent des stories qui changent en cours de sprint parce que « le métier a réalisé que c'était pas exactement ça »
- Le QA ne peut pas écrire de tests d'acceptation parce que les critères d'acceptation sont absents ou ambigus
- La démonstration de fin de sprint génère des surprises chez les parties prenantes (« ah, c'est comme ça que vous l'avez compris ? »)
- Les estimations d'effort sont systématiquement sous-estimées parce que la complexité réelle n'avait pas été explorée lors de l'élicitation
- Les bugs de production touchent des cas d'usage que personne n'avait mentionnés — parce que personne n'avait demandé

❓ **Questions de réflexion**

- Lors de votre dernière élicitation, avez-vous passé plus de temps à écouter ou à présenter ?
- Avez-vous observé l'utilisateur dans son contexte réel, ou uniquement recueilli sa description de ce contexte ?
- Combien de fois avez-vous posé « et derrière ça, qu'est-ce qui se passe vraiment ? » ?
- À la fin de l'élicitation, toute l'équipe (PO, tech lead, QA) avait-elle la même compréhension du besoin ?
- Auriez-vous pu formuler, en une phrase, la douleur principale — sans

mentionner de solution ?

🎭 Scène ORIGINALIUM — La douleur cachée derrière la demande

Théo avait convoqué l'atelier d'élicitation pour 14 h. La demande sur la table était simple, du moins en apparence : « Ajouter un filtre de recherche avancée dans BEASTEST. » La direction produit avait déjà spécifié le filtre en gros — par type de test, par statut, par date d'exécution. Théo avait prévu 45 minutes pour « confirmer la portée ».

Rania avait apporté son carnet. Karim avait apporté ses questions.

Les 20 premières minutes s'étaient passées comme prévu. Théo présentait les filtres envisagés, Rania posait des questions d'architecture, Karim prenait des notes sur la testabilité. Et puis Karim avait posé une question qui n'était pas dans le script :

— « Attends. Pourquoi les gens cherchent dans les tests en ce moment ? »

Silence. Personne n'avait posé cette question-là.

Théo avait ouvert Slack et fouillé les messages. Un pattern était apparu : les utilisateurs cherchaient des tests spécifiques parce qu'ils n'arrivaient pas à comprendre pourquoi un test avait échoué la veille, et ils voulaient retrouver l'exécution précise pour voir les logs. Le problème n'était pas l'absence de filtre avancé. Le problème était l'absence d'un accès rapide aux logs d'exécution liés à chaque test.

Rania avait posé son stylo : « On était sur le point de construire la mauvaise chose. »

Le filtre de recherche pouvait faciliter et accélérer la navigation — BEASTEST offrait effectivement cette capacité. Mais la vraie douleur était ailleurs. Elle n'avait jamais été nommée, parce que personne ne l'avait cherchée. La demande de fonctionnalité était une réponse à un symptôme — pas au problème sous-jacent.

Théo avait rouvert son ordinateur et changé le titre du backlog item : « Accès rapide aux logs d'exécution depuis la liste de tests. » L'atelier avait duré encore 40 minutes. L'équipe était repartie avec quelque chose de précieux : une vraie douleur, bien nommée.

📓 Carnet de terrain — Quand la véritable douleur se révèle derrière la demande initiale

J'ai vécu des dizaines de versions de cette scène. Une demande arrive, bien formulée, apparemment claire. L'équipe commence à travailler dessus. Et à mi-chemin — parfois en sprint review, parfois directement en production — quelqu'un dit : « En fait, le vrai problème, c'est pas ça. »

La tentation, à ce moment, est de blâmer les parties prenantes : elles n'ont pas su exprimer leur besoin. Mais c'est rarement juste. La plupart du temps, les parties prenantes ont exprimé ce qu'elles pouvaient exprimer avec les mots qu'elles avaient. Le problème, c'est qu'on ne leur a pas posé les bonnes questions. On n'a pas creusé assez loin. On n'a pas demandé « pourquoi » une troisième fois.

La véritable douleur se cache rarement dans la première réponse. Elle se trouve dans la conversation qui s'ouvre après que vous ayez posé la question que tout le monde avait évitée parce qu'elle semblait trop basique, trop évidente, ou trop inconfortable. La prochaine fois qu'une demande vous semble parfaitement claire, posez-vous la question : « Ai-je cherché la douleur réelle, ou ai-je accepté la demande de surface ? »

Chapitre 2 — ÉLICITER en pratique : les 5 profils de sources de besoin

SENSE · Phase 1 · En pratique

L'élicitation ne se pratique pas dans l'abstrait. Elle se pratique avec des sources — des personnes, des systèmes, des environnements — qui détiennent chacun une fraction de la vérité sur le besoin. Savoir où chercher est aussi important que savoir comment chercher.

Dans le SDLC réconcilié, on identifie cinq grands profils de sources de besoin. Chaque profil apporte une perspective différente, génère des types de signaux différents, et comporte des risques spécifiques si on l'ignore.

Profil 1 — L'utilisateur direct

L'utilisateur direct est la personne qui touche le produit au quotidien. Elle ouvre l'interface, clique sur les boutons, entre les données, lit les résultats. Elle est la source la plus immédiate des signaux de douleur et de valeur — et paradoxalement, souvent la source la moins bien écoutée.

Le risque de l'utilisateur direct est le risque de la saturation : on pense le connaître parce qu'on l'a déjà rencontré, parce qu'on a déjà fait des entretiens utilisateurs, parce qu'on a des personas bien documentés. Mais les besoins de l'utilisateur direct évoluent — avec les versions du produit, avec les changements organisationnels, avec les nouvelles habitudes de travail. Une élicitation qui s'appuie sur des personas périmés est une élicitation dans le vide.

Ce que l'utilisateur direct révèle mieux que quiconque : les micro-frustrations quotidiennes, les contournements créatifs, les cas d'usage non prévus, les patterns d'abandon.

Profil 2 — Le commanditaire

Le commanditaire est la personne ou l'entité qui finance le développement et attend de la valeur en retour. Il peut être un client externe, un sponsor interne, une direction. Sa perspective est radicalement différente de celle de l'utilisateur direct : il raisonne en termes d'objectifs stratégiques, de retour sur investissement, de risques organisationnels.

Le risque du commanditaire est le risque de la déconnexion : il peut formuler des demandes qui sont cohérentes avec sa vision stratégique mais déconnectées de la réalité opérationnelle des utilisateurs. Il peut aussi formuler des objectifs tellement larges qu'ils deviennent intraduisibles en spécifications concrètes.

Ce que le commanditaire révèle mieux que quiconque : les contraintes non dites, les enjeux politiques, les priorités réelles (souvent différentes des priorités affichées), les critères de succès à long terme.

Profil 3 — Le système

Le système est une source de besoin non humaine : les incidents en production, les métriques d'usage, les logs d'erreurs, les patterns de performance, les alertes de monitoring. C'est la source que les équipes techniques connaissent le mieux théoriquement — et qu'elles exploitent le moins dans l'élicitation.

Traiter les données du système comme une source d'élicitation, c'est admettre que le logiciel lui-même parle : ses métriques d'usage disent quelles fonctionnalités sont réellement utilisées, ses logs d'erreurs révèlent quels cas limites n'ont pas été anticipés, ses patterns de performance indiquent où les utilisateurs abandonnent.

Ce que le système révèle mieux que quiconque : les comportements réels (versus les comportements déclarés), les angles morts de la conception, les dégradations progressives de valeur, les besoins d'optimisation cachés.

Profil 4 — Le marché

Le marché est la source la plus externe : les concurrents, les tendances d'industrie, les évolutions réglementaires, les innovations émergentes. C'est une source

indispensable pour les produits qui évoluent dans un écosystème compétitif — et une source négligée dans les contextes purement internes, parfois à tort.

Ce que le marché révèle mieux que quiconque : les standards implicites d'expérience que les utilisateurs ont intégrés ailleurs, les changements réglementaires qui vont contraindre les choix techniques, les opportunités différenciatrices que la concurrence n'a pas encore saisies.

Profil 5 — L'équipe

L'équipe est une source de besoin souvent ignorée — à tort. Les développeurs, les QA, les ops qui travaillent chaque jour sur le système accumulent une connaissance intime de ses faiblesses, de sa dette technique, de ses fragilités architecturales. Ils voient des besoins que personne d'autre ne peut voir parce qu'ils sont au plus près du moteur.

Les signaux internes de l'équipe incluent : la dette technique qui ralentit chaque itération, les modules qui concentrent le plus de corrections, les processus de déploiement qui consomment le plus de temps humain, les tests qui cassent trop souvent sans raison claire.

Ce que l'équipe révèle mieux que quiconque : les besoins d'ingénierie qui conditionnent la qualité à long terme, les risques techniques implicites, les opportunités d'amélioration interne à haute valeur.

Grille de synthèse : les 5 profils

Profil	Type de signal	Question clé	Risque si ignoré
Utilisateur direct	Friction, douleur opérationnelle, usage réel	Qu'est-ce qui vous ralentit ou vous frustre chaque jour ?	Produit techniquement correct mais inutilisable en contexte réel
Commanditaire	Valeur stratégique, contraintes organisationnelles	Sur quels critères allez-vous juger que ce projet est un succès dans 12	Livraison qui satisfait l'opérationnel mais manque l'objectif

Profil	Type de signal	Question clé	Risque si ignoré
		mois ?	stratégique
Système	Comportement réel, dégradations, angles morts	Que disent les métriques d'usage sur ce que les utilisateurs font réellement ?	Ignorance des problèmes réels au profit des problèmes déclarés
Marché	Standards, concurrence, régulation	Quelles évolutions extérieures vont contraindre ou enrichir nos choix dans les 18 prochains mois ?	Surprises réglementaires tardives, perte de compétitivité silencieuse
Équipe	Dette technique, fragilités, opportunités d'ingénierie	Qu'est-ce qui ralentit le plus votre capacité à livrer avec qualité ?	Accumulation de dette invisible jusqu'à la crise technique

Comment prioriser les sources selon le contexte

Toutes les sources ne sont pas également importantes dans tous les contextes. La priorité relative des profils dépend de la nature du projet, de son stade de maturité, et de la nature de la douleur principale.

Voici quelques règles de priorisation pratiques :

- **Projet en phase de découverte initiale :** Priorité à l'utilisateur direct et au commanditaire. On cherche à comprendre le terrain humain avant tout.
- **Produit mature en amélioration continue :** Priorité au système et à l'équipe. Les données réelles d'usage et la dette interne révèlent ce que les entretiens ne disent plus.
- **Contexte réglementaire ou compétitif en mutation :** Priorité au marché. Les signaux extérieurs conditionnent toutes les décisions internes.
- **Projet à forte composante technique :** Ne pas négliger l'équipe — ses signaux internes sont souvent les plus actionables à court terme.

- **Quand les parties prenantes ne s'accordent pas :** Retourner vers le système. Les données réelles tranchent souvent des désaccords que les opinions ne résolvent pas.

> 💡 **À retenir — Les 5 profils**
>
> Une élicitation robuste ne se limite pas à une seule source. Elle triangule entre les 5 profils pour construire une compréhension multidimensionnelle du besoin. La douleur de l'utilisateur direct, la valeur du commanditaire, les signaux du système, les contraintes du marché, et les fragilités de l'équipe — ensemble, ces cinq perspectives forment un portrait complet du territoire à traiter.

> ⚠ **Signal de vigilance — La source unique**
>
> Une élicitation qui ne consulte qu'une seule source — le plus souvent, le commanditaire ou le PO seul — produit un biais systématique vers les besoins les mieux articulés au détriment des besoins les plus importants. Le commanditaire sait ce qu'il veut. Il ne sait pas toujours ce dont les utilisateurs ont besoin, ni ce que le système lui dit en silence.

> ❓ **Questions de réflexion**
>
> • Pour votre dernier projet, combien de sources différentes avez-vous consultées lors de l'élicitation ?
>
> • Avez-vous déjà regardé les logs de votre système comme une source de besoin — pas seulement comme un outil de debugging ?
>
> • Quand avez-vous dernièrement demandé à un développeur ou un QA : « Qu'est-ce qui vous ralentit le plus en ce moment ? »

- Votre compréhension du marché date de quand ? Les standards implicites de vos utilisateurs ont-ils changé depuis ?

Chapitre 3 — DÉFINIR : transformer le besoin en spécification observable

SENSE · Phase 2

Si ÉLICITER est l'art de comprendre, DÉFINIR est l'art de formuler. Et formuler est plus difficile que comprendre. Comprendre, c'est un état intérieur. Formuler, c'est externaliser cet état de façon à ce qu'une autre personne — un développeur, un testeur, un utilisateur — puisse en vérifier la réalité dans un artefact observable.

C'est ici que la plupart des équipes échouent. Pas parce qu'elles manquent de méthode. Parce qu'elles confondent « avoir compris » avec « avoir défini ». Ces deux choses sont fondamentalement différentes. On peut comprendre un besoin parfaitement et être totalement incapable de le formuler de façon testable. Et c'est précisément cette incapacité qui crée la fracture entre ce qui a été voulu et ce qui a été livré.

DÉFINIR n'est pas « écrire des user stories »

La user story — format canonique de l'Agile — est une technique utile. « En tant que [rôle], je veux [action] afin de [bénéfice]. » C'est une façon de centrer la conversation sur l'utilisateur, de relier une fonctionnalité à un besoin humain. C'est précieux.

Mais la user story n'est pas une spécification. C'est une invitation à spécifier. Elle dit : « Il y a quelqu'un, qui veut quelque chose, pour une raison. » Elle ne dit pas : « Voici le

comportement précis, observable, testable que le système devra exhiber pour que ce quelqu'un obtienne ce quelque chose. »

Dans le SDLC réconcilié, DÉFINIR va au-delà de la user story. Elle produit des spécifications — des énoncés qui rendent le comportement attendu du système suffisamment précis pour être vérifiable. Une user story sans spécification est une promesse sans contrat.

La spécification ToTAL : l'attente rendue observable et testable

Une spécification ToTAL est un énoncé qui satisfait trois propriétés simultanées :

1. **Observable :** Le résultat décrit peut être observé par une personne ou un système externe — il n'est pas interne à l'implémentation.
2. **Testable :** Il est possible de concevoir un test — manuel ou automatisé — qui vérifie que le résultat a été atteint.
3. **Non ambigu :** Deux personnes différentes lisant la spécification arrivent à la même compréhension de ce qui est attendu.

Ces trois propriétés ne sont pas des idéaux — ce sont des critères de seuil. Une spécification qui ne les satisfait pas n'est pas une spécification imparfaite : c'est quelque chose d'autre — une intention, une hypothèse, un souhait. Et les intentions ne se testent pas.

La forme canonique : SI [condition observable] ALORS [résultat observable attendu]

Le SDLC réconcilié utilise une forme canonique pour les spécifications : la structure SI/ALORS. Elle est simple. Elle est puissante. Et elle force précisément les deux niveaux d'observabilité qu'une spécification doit couvrir.

> SI [condition observable] ALORS [résultat observable attendu]

La partie **SI** décrit le contexte et le déclencheur : quelle est la situation, qui fait quoi, dans quel état le système se trouve. La partie **ALORS** décrit le comportement attendu du système : qu'est-ce qui change, qu'est-ce qui apparaît, qu'est-ce qui se produit.

Voici des exemples concrets pour illustrer la puissance et la rigueur de cette forme :

Énoncé vague	Spécification SI/ALORS
« L'utilisateur doit pouvoir rechercher des tests facilement »	SI l'utilisateur saisit un mot-clé dans le champ de recherche et appuie sur Entrée, ALORS la liste des tests affichés est filtrée pour ne montrer que les tests dont le nom ou la description contient ce mot-clé, dans un délai inférieur à 500 ms.
« Le système doit gérer les erreurs de connexion »	SI l'utilisateur soumet le formulaire de connexion avec un mot de passe incorrect, ALORS le système affiche le message « Identifiants incorrects. Veuillez réessayer. » sans révéler si c'est l'identifiant ou le mot de passe qui est erroné, et le formulaire reste actif.
« L'exportation doit fonctionner »	SI l'utilisateur clique sur « Exporter en CSV » avec une liste de 0 à 10 000 résultats affichés, ALORS le fichier CSV est téléchargé dans le navigateur dans un délai inférieur à 3 secondes, avec une ligne d'en-tête et une ligne par résultat.

Pourquoi SI/ALORS : la logique naturelle humaine appliquée au logiciel

La structure SI/ALORS n'a pas été inventée pour le logiciel. C'est la structure fondamentale du raisonnement conditionnel humain — celle que tout être humain apprend à utiliser naturellement dès l'enfance. « Si tu traverses la rue sans regarder, alors il peut t'arriver quelque chose. » « Si tu soumets ta demande avant vendredi, alors elle sera traitée cette semaine. »

Son application au logiciel est donc intuitive — pour le développeur qui implémente, pour le testeur qui vérifie, et pour l'utilisateur qui valide. Une spécification SI/ALORS

est lisible par tout le monde, sans formation spécialisée. C'est précisément sa force : elle crée un langage partagé qui traverse les frontières fonctionnelles et techniques.

Elle force aussi deux honnêtetés nécessaires. La condition du SI force à préciser le contexte — on ne peut pas écrire un SI sans se demander : dans quelle situation exactement ? La conséquence du ALORS force à préciser le résultat — on ne peut pas écrire un ALORS sans se demander : qu'est-ce qui sera différemment observable ?

Une spécification atomique

Le principe d'atomicité est fondamental dans la définition ToTAL. Une spécification atomique est une spécification qui couvre exactement :

- **Une condition** (une situation, un déclencheur)
- **Un résultat** (un comportement, une sortie)
- **Un niveau d'observation** (visible dans l'interface ? dans les logs ? dans la base de données ?)
- **Une preuve possible** (comment peut-on vérifier ce résultat ?)

Quand une spécification couvre plusieurs conditions ou plusieurs résultats, elle doit être décomposée. La décomposition n'est pas une perte de temps — c'est un gain de clarté et de testabilité. Chaque spécification atomique correspond à un test potentiellement écrit. Une spécification qui couvre cinq comportements en une seule phrase génère un test qui ne peut pas être automatisé proprement.

Le piège des spécifications décoratives

Les spécifications décoratives sont des énoncés qui ressemblent à des spécifications mais qui n'en sont pas. Elles consomment du temps sans réduire l'incertitude — et pire, elles donnent l'illusion d'avoir défini quelque chose alors qu'on n'a fait que reformuler la demande initiale avec plus de mots.

Voici les formes les plus fréquentes de spécification décorative :

- « Le système doit être performant. » → Performant comment ? Dans quel contexte ? Mesurable comment ?

- « L'interface doit être intuitive. » → Intuitif pour qui ? Vérifié comment ? Par rapport à quel standard ?
- « Les données doivent être sécurisées. » → Quel niveau de sécurité ? Quel type de menace ? Certifié comment ?
- « Le module doit gérer les cas d'erreur. » → Quels cas d'erreur ? Avec quel comportement attendu pour chacun ?

Ces énoncés sont confortables parce qu'ils semblent raisonnables. Tout le monde acquiesce quand on les lit. Le problème apparaît quand le développeur doit les implémenter (lequel des 10 niveaux de performance visait-on ?) ou quand le QA doit les tester (comment vérifie-t-on que c'est « intuitif » ?). Les spécifications décoratives sont des bombes à retardement d'ambiguïté.

> ⚠ **Signal de vigilance — Spécification décorative**
>
> Testez chaque spécification avec la question : « Comment Karim va-t-il écrire un test qui vérifie cet énoncé ? » Si la réponse est « il ne peut pas — il faudrait d'abord décider ce que ça veut dire concrètement », alors la spécification est décorative. Recommencez.

Critère de sortie de DÉFINIR : la DoD-spec

La phase DÉFINIR se termine lorsque les spécifications produites satisfont un ensemble de critères convenus — la **Definition of Done spécification (DoD-spec)**. Comme la DoR, la DoD-spec n'est pas une déclaration de perfection — c'est une déclaration de suffisance pour entrer en PLANIFIER.

#	Critère DoD-spec	Question de validation
1	Forme SI/ALORS respectée	Chaque spécification est-elle formulée avec une condition observable et un résultat observable ?
2	Atomicité vérifiée	Chaque spécification

#	Critère DoD-spec	Question de validation
		couvre-t-elle exactement une condition et un résultat ?
3	Testabilité confirmée	Karim (QA) peut-il écrire un test pour chaque spécification telle qu'elle est formulée ?
4	Lecture partagée	Rania (tech lead), Théo (PO) et Karim (QA) lisent-ils la même chose dans chaque spécification ?
5	Traçabilité au besoin	Chaque spécification peut-elle être reliée à un besoin identifié lors de l'élicitation ?

🎭 **Scène ORIGINALIUM — « Comment je teste ça ? »**

C'était le troisième sprint de planification du trimestre. Théo avait passé la semaine à rédiger les stories. Rania avait revu l'architecture. Et Karim avait enfin eu le temps de lire le backlog au complet.

Karim avait posé son téléphone sur la table et regardé Théo avec cette expression tranquille mais ferme qu'on lui connaissait quand quelque chose ne tournait pas rond.

— « Story numéro 47. "L'utilisateur doit pouvoir gérer ses notifications facilement." Comment je teste ça ? »

Théo avait ouvert la story sur son écran. « Ben… tu vérifies que les notifications s'affichent, que l'utilisateur peut les désactiver, que… »

— « "Facilement." C'est quoi, facilement ? En combien de clics ? Dans quel délai ? Avec quel résultat visible ? »

Silence.

Rania avait souri. Elle avait vu cette scène des dizaines de fois. Pas parce que Théo était mauvais — il était excellent. Mais parce que la zone entre «

comprendre un besoin » et « le formuler de façon testable » est une zone de transformation difficile que tout le monde doit apprendre à traverser.

Ils avaient passé les 20 minutes suivantes à réécrire la story. La version finale était en trois spécifications SI/ALORS distinctes : une pour l'activation, une pour la désactivation, une pour le comportement en cas de notifications en attente lors de la désactivation. Chacune était testable. Chacune était sans ambiguïté.

Karim avait hoché la tête. « Maintenant je peux travailler. »

Rania avait ajouté, pour Théo : « La question de Karim, c'est la meilleure question de qualité qu'une équipe puisse poser. Si on peut pas y répondre, la spécification est pas finie. »

💡 À retenir — DÉFINIR

DÉFINIR transforme le besoin compris en spécification observable et testable. La forme canonique SI/ALORS est le véhicule de cette transformation. Une spécification qui ne peut pas être testée telle qu'elle est formulée n'est pas une spécification — c'est une intention. La question « Comment je teste ça ? » posée par le QA est le meilleur vérificateur de qualité d'une spécification.

🔍 Symptômes à observer — DÉFINIR mal faite

- Les critères d'acceptation sont écrits après la démonstration, non avant l'implémentation
- Le QA découvre des ambiguïtés en phase de test — pas en phase de définition
- Les développeurs prennent des décisions de conception par défaut parce que la spécification ne les guidait pas

- Les stories sont « terminées » mais les parties prenantes les rejettent en review
- La même fonctionnalité est implémentée différemment par deux développeurs différents parce que la spécification admettait deux lectures valides

❓ **Questions de réflexion**

- Pour votre dernière story livrée, existait-il une spécification SI/ALORS avant que l'implémentation commence ?
- Vos critères d'acceptation sont-ils écrits avant ou après l'implémentation ?
- Combien de vos spécifications actuelles contiennent des termes subjectifs non mesurables (« performant », « intuitif », « robuste », « facile ») ?
- Vos développeurs et votre QA ont-ils la même compréhension de vos spécifications — ou chacun interprète-t-il différemment ?

Chapitre 4 — DÉFINIR en pratique : niveaux et types de spécification

SENSE · Phase 2 · En pratique

Dans la pratique, toutes les spécifications ne sont pas au même niveau de granularité. Un produit logiciel est un empilement de couches — des objectifs stratégiques jusqu'aux comportements précis de l'interface — et chaque couche requiert un type

de spécification adapté. Comprendre cette hiérarchie, c'est comprendre comment le travail de définition se structure du macro au micro.

Niveaux de granularité : du portefeuille au comportement

Le SDLC réconcilié organise la spécification en trois niveaux de granularité :

Niveau	Unité	Question	Exemple	Horizon
Portefeuille	Épique / Feature	Quelle capacité métier voulons-nous créer ?	« Permettre aux équipes QA de filtrer et d'analyser les résultats de test en temps réel »	Trimestre / Année
Produit	Story / Capability	Quel comportement utilisateur voulons-nous rendre possible ?	« L'utilisateur peut filtrer la liste des tests par statut (réussi, échoué, ignoré) »	Sprint / Mois
Comportement	Scénario SI/ALORS	Quel comportement précis du système voulons-nous vérifier ?	« SI l'utilisateur sélectionne "échoué" dans le filtre de statut, ALORS seuls les tests avec statut "échoué" s'affichent, dans un délai < 300 ms. »	Tâche / Jour

Ces trois niveaux ne sont pas indépendants — ils sont hiérarchiquement connectés. Une épique se décompose en stories. Une story se décompose en scénarios SI/ALORS. La traçabilité entre niveaux est ce qui permet de s'assurer qu'on ne perd jamais de vue pourquoi on construit quelque chose — même quand on est au niveau le plus granulaire.

Types de spécification dans ToTAL

La granularité dit *à quel niveau* on spécifie. Le type dit *ce qu'on spécifie*. Le SDLC réconcilié distingue quatre grands types de spécification :

1. La spécification fonctionnelle

Elle décrit ce que le système fait : ses comportements visibles, ses traitements, ses entrées et sorties. C'est le type le plus courant, et celui pour lequel la forme SI/ALORS est la plus naturelle. « SI l'utilisateur soumet le formulaire avec tous les champs obligatoires remplis, ALORS le système crée l'enregistrement et affiche un message de confirmation. »

2. La spécification non fonctionnelle

Elle décrit comment le système se comporte : sa performance, sa sécurité, sa fiabilité, son accessibilité. La spécification non fonctionnelle est souvent la plus négligée — et la plus coûteuse à corriger tard. Elle doit aussi être formulée en SI/ALORS observable. « SI le tableau de bord est chargé avec 50 000 enregistrements, ALORS le temps de chargement de la page est inférieur à 2 secondes dans un environnement de production standard. »

3. La spécification d'acceptation

Elle décrit les conditions dans lesquelles les parties prenantes valident que la valeur a été créée. C'est la spécification qui fait le lien entre la vision du commanditaire et les comportements observables du produit. Elle s'exprime souvent en termes de résultats métier plutôt que de comportements systèmes. « SI le module de reporting est déployé, ALORS l'équipe QA est capable de produire son rapport hebdomadaire en moins de 15 minutes sans assistance technique. »

4. La spécification d'apprentissage

C'est le type le moins connu, et l'un des plus puissants. Il s'agit d'une spécification qui définit ce qu'on cherche à apprendre — non pas ce qu'on cherche à construire. Elle est particulièrement utile dans les contextes d'incertitude élevée, où on ne sait pas encore si une solution va fonctionner. « SI nous déployons la fonctionnalité X auprès

de 10 % des utilisateurs pendant deux semaines, ALORS nous observerons si le taux d'abandon du formulaire d'inscription diminue d'au moins 15 %. »

La spécification d'apprentissage est la passerelle entre DÉFINIR et la grande boucle : elle prépare dès la phase de définition ce que la phase CONFIRMER devra mesurer.

La traçabilité ToTAL : du signal au besoin jusqu'à la preuve

Dans le SDLC réconcilié, chaque spécification doit être traçable — vers le besoin dont elle est issue, et vers le test qui la vérifiera. Cette double traçabilité est le garant de la cohérence du cycle.

> Signal de besoin (ÉLICITER) → Besoin nommé → Spécification SI/ALORS (DÉFINIR) → Test de vérification (CONFIRMER)

La traçabilité n'est pas un exercice administratif. C'est une discipline de qualité. Quand un test échoue en phase de test ou en production, la traçabilité permet de remonter immédiatement à la spécification concernée, puis au besoin dont elle est issue. Quand un besoin change, la traçabilité permet d'identifier toutes les spécifications et tous les tests impactés — sans avoir à fouiller l'ensemble du backlog à la main.

Spécification et testabilité : la dette cachée

Il existe une forme de dette logicielle dont on parle rarement : la dette de spécification. Un test qui ne peut pas être lié à une spécification est un test orphelin. Un test orphelin est une forme de dette — il coûte du temps à écrire, du temps à maintenir, et il ne contribue pas à la confiance dans la valeur livrée parce qu'on ne sait pas ce qu'il est censé prouver.

Symétriquement, une spécification sans test correspondant est une spécification non vérifiée — ce qui signifie qu'on ne sait pas si le comportement qu'elle décrit existe

réellement dans le produit livré. C'est une autre forme de dette : la dette de vérification.

Le SDLC réconcilié traite la testabilité comme une propriété de première classe de la spécification — non comme une propriété du test. Si une spécification n'est pas testable telle qu'elle est formulée, c'est un problème de définition, pas de QA.

La spécification comme contrat lisible

Une bonne spécification ToTAL est un contrat lisible. Elle n'est pas un document technique destiné aux seuls développeurs. Elle n'est pas un document métier incompréhensible pour les techniciens. Elle est un langage commun — lu et compris de la même façon par le PO (Théo), la tech lead (Rania), le QA (Karim), et le représentant métier qui validera la livraison.

Cette lisibilité universelle n'est pas un luxe — c'est une condition de confiance. Une équipe qui partage le même langage de spécification réduit drastiquement les malentendus en cours de sprint, les surprises en démonstration, et les désaccords en validation. Elle construit quelque chose de précieux : la capacité de vérifier ensemble, à tout moment, si ce qui a été décidé correspond à ce qui a été construit.

📓 Carnet de terrain — Le vertige de dire ce que le produit doit faire

Dans une grande entreprise technologique spécialisée en systèmes embarqués temps réel, j'ai travaillé plusieurs mois avec une équipe exceptionnellement talentueuse sur le plan technique. Des ingénieurs brillants, des architectures élégantes, une culture de rigueur impressionnante. Et pourtant, leurs spécifications étaient catastrophiques.

Pas par manque de compétence. Par excès de confort technique. Ces ingénieurs savaient exactement comment ils allaient implémenter chaque fonctionnalité. Et parce qu'ils savaient comment, ils n'avaient jamais eu besoin de formuler clairement quoi. L'implémentation était leur spécification.

Le problème est apparu en revue de produit, quand les clients ont commencé à utiliser les systèmes livrés. Non pas que les systèmes ne fonctionnaient pas — ils

fonctionnaient parfaitement selon les critères internes de l'équipe. Mais les clients avaient une définition différente de « fonctionner ». Une définition que personne n'avait jamais prise le temps de formuler explicitement, de vérifier ensemble, de valider avant que la construction commence.

C'est ce que j'appelle le vertige de dire ce que le produit doit faire. Quand on sait déjà comment on va le faire, formuler explicitement ce qu'il doit faire semble superflu. Mais ce superflu apparent est la condition de tous les alignements ultérieurs. La spécification n'est pas pour ceux qui construisent — c'est pour ceux qui valident. Et ceux qui valident ne voient jamais le code. Ils voient le comportement.

💡 **À retenir — Niveaux et types de spécification**

La spécification ToTAL opère sur trois niveaux (portefeuille, produit, comportement) et couvre quatre types (fonctionnel, non fonctionnel, acceptation, apprentissage). La traçabilité entre niveaux et types garantit la cohérence du cycle. La testabilité est une propriété de la spécification — pas du test. Un test orphelin et une spécification non vérifiée sont deux formes de dette silencieuse.

⚠ **Signal de vigilance — Spécification non fonctionnelle négligée**

Les spécifications non fonctionnelles — performance, sécurité, fiabilité, accessibilité — sont les plus fréquemment omises en phase DÉFINIR. Elles réapparaissent en production, souvent au pire moment. Tout projet doit avoir au moins une conversation explicite sur les seuils non fonctionnels critiques avant d'entrer en FORGE.

❓ **Questions de réflexion**

- Vos spécifications sont-elles lisibles par quelqu'un qui ne connaît pas l'implémentation technique ?
- Avez-vous des spécifications non fonctionnelles explicites pour la performance et la sécurité ?
- Pouvez-vous tracer chaque test de votre suite de tests jusqu'à une spécification précise ?
- Avez-vous déjà écrit une spécification d'apprentissage — qui définit ce que vous cherchez à découvrir plutôt que ce que vous cherchez à construire ?

Chapitre 5 — SENSE en pratique : conduire l'atelier ÉLICITER-DÉFINIR

SENSE · Zone complète · En pratique

Les chapitres précédents ont exploré ÉLICITER et DÉFINIR séparément. Dans la pratique, ces deux phases s'articulent dans un moment de travail collectif que le SDLC réconcilié formalise : l'**atelier SENSE**. Cet atelier n'est pas un rituel bureaucratique — c'est un moment de création de connaissance partagée, structuré pour produire des outputs concrets et exploitables.

Un atelier SENSE bien conduit est l'un des investissements les plus rentables qu'une équipe puisse faire. Il évite des semaines de malentendus en aval. Il aligne les parties prenantes sur ce qui compte vraiment. Et il produit des spécifications assez solides pour que l'équipe entre en FORGE avec confiance.

L'atelier SENSE ToTAL : déroulement en 3 temps

Temps 1 — Comprendre (ÉLICITER)

Le premier temps de l'atelier est consacré à l'exploration du territoire humain. L'objectif : construire une compréhension partagée du besoin, de la douleur, et du contexte. Ce temps dure typiquement 40 à 60 minutes pour un besoin de complexité moyenne.

Activités typiques du Temps 1 :

- Présentation du signal de départ : qui l'a émis, dans quel contexte, sous quelle forme
- Exploration des 5 objets d'élicitation (douleur, valeur, risque, décision, signal) en groupe
- Carte d'empathie collective si le besoin est centré sur un utilisateur spécifique
- Formulation collaborative de la douleur principale en une phrase sans solution

Sortie du Temps 1 : la douleur principale est nommée et validée par tous les participants. La DoR est vérifiée colonne par colonne.

Temps 2 — Nommer (transition)

Le deuxième temps est le moment de transition — souvent le plus court (15 à 20 minutes), mais le plus critique. Il s'agit de passer de la compréhension du besoin à sa formulation en unités spécifiables.

Activités typiques du Temps 2 :

- Identification des comportements clés à spécifier (« qu'est-ce que le système doit faire différemment ? »)
- Décomposition en niveaux : quelle épique ? quelles stories ? quels comportements ?

- Priorisation initiale : quelles spécifications sont critiques pour une première livraison de valeur ?

Sortie du Temps 2 : une liste de candidats à spécifier, priorisés et prêts à entrer dans le Temps 3.

Temps 3 — Spécifier (DÉFINIR)

Le troisième temps est le travail de définition lui-même. Les candidats identifiés au Temps 2 sont formulés en spécifications SI/ALORS par l'équipe, en collaboration. Ce temps dure typiquement 60 à 90 minutes selon le nombre de spécifications à produire.

Activités typiques du Temps 3 :

- Rédaction des spécifications SI/ALORS en groupe (tableau blanc, outil collaboratif, ou BEASTEST si disponible et utilisé)
- Challenge de testabilité : Karim valide chaque spécification avec la question « Comment je teste ça ? »
- Vérification de l'atomicité : chaque spécification est-elle décomposée assez finement ?
- Vérification de la DoD-spec : les 5 critères sont-ils satisfaits ?

Sortie du Temps 3 : un ensemble de spécifications DoD-spec-validées, prêtes à entrer en phase PLANIFIER (FORGE).

Outils de l'atelier SENSE ToTAL

Outil	Temps	Objectif	Format
Carte de signal	Temps 1	Documenter le signal de départ : source, nature, contexte	Fiche structurée (papier ou numérique)
Grille d'élicitation	Temps 1	Explorer les 5 objets (douleur, valeur, risque, décision, signal) de	Tableau à 5 colonnes, rempli en groupe

Outil	Temps	Objectif	Format
		façon structurée	
Carte d'empathie	Temps 1	Construire une représentation partagée de l'expérience utilisateur	Canevas à 6 zones (dit, pense, ressent, fait, entend, voit)
Canvas de spécification	Temps 3	Rédiger, valider et documenter les spécifications SI/ALORS	Tableau : condition | résultat | niveau d'observation | preuve
DoR Checklist	Fin Temps 1	Valider que le besoin est prêt à être défini	5 critères à valider colonne par colonne
DoD-spec Checklist	Fin Temps 3	Valider que les spécifications sont prêtes à entrer en FORGE	5 critères à valider colonne par colonne

Ces outils peuvent être utilisés avec n'importe quel support — tableau blanc et Post-it, document partagé, ou outil dédié. BEASTEST peut faciliter et accélérer la gestion de ces outils dans les équipes qui y ont accès, notamment pour la traçabilité des spécifications et la gestion des DoR/DoD-spec. Mais chaque outil reste pleinement utilisable sans plateforme dédiée.

Rôles en atelier SENSE

L'atelier SENSE réunit plusieurs rôles, chacun avec une contribution spécifique :

Rôle	Contribution principale	Moment critique
Facilitateur	Guide le déroulement, gère le temps, protège la qualité du dialogue	Tout l'atelier
Product Owner (Théo)	Apporte la vision produit, connecte les besoins aux objectifs stratégiques, priorise	Temps 1 (douleur et valeur) et Temps 2 (priorisation)
Tech Lead (Rania)	Évalue la faisabilité, identifie les contraintes	Temps 1 (risque) et Temps 3 (observabilité technique)

Rôle	Contribution principale	Moment critique
	techniques, soulève les risques d'implémentation	
QA (Karim)	Challenge la testabilité de chaque spécification, identifie les cas limites non couverts	Temps 3 (validation testabilité)
Représentant métier	Ancre les discussions dans la réalité opérationnelle, valide la pertinence des spécifications	Temps 1 (douleur réelle) et Temps 3 (validation acceptation)

Anti-patterns SENSE : ce qu'il faut éviter absolument

L'atelier SENSE peut échouer de plusieurs façons typiques. Ces anti-patterns sont suffisamment fréquents pour mériter d'être nommés explicitement.

Anti-pattern 1 — L'élicitation superficielle

On passe 10 minutes à « confirmer » un besoin qui avait été formulé à l'avance par le commanditaire, sans explorer les 5 objets, sans consulter les 5 profils de sources, sans poser les questions inconfortables. On sort de l'atelier avec l'impression d'avoir « compris » — mais on a surtout validé une hypothèse initiale sans la challenger.

Remède : Interdire les affirmations les 20 premières minutes. L'atelier commence en mode question — seulement des questions. Les affirmations viennent après.

Anti-pattern 2 — La définition sans validation

On produit des spécifications rapidement, mais elles ne sont lues que par celui qui les écrit. La validation croisée (le QA challenge, la tech lead vérifie, le représentant métier confirme) est sautée par manque de temps. Les spécifications sortent de l'atelier non challengées et fragiles.

Remède : La règle des trois lectures. Chaque spécification doit être lue à voix haute par quelqu'un autre que celui qui l'a écrite. Si la lecture génère une question, la spécification est reécrite avant de sortir de la pièce.

Anti-pattern 3 — Le rush vers FORGE

La pression de livraison est telle qu'on coupe court à SENSE pour « commencer à construire ». On entre en PLANIFIER avec des spécifications incomplètes, en se disant qu'on « clarifiera en cours de route ». En cours de route, les clarifications génèrent des réorientations, les réorientations génèrent des refontes, et les refontes coûtent infiniment plus cher que 30 minutes supplémentaires dans l'atelier SENSE.

Remède : La DoR et la DoD-spec comme portes de passage. Si les critères ne sont pas satisfaits, l'atelier n'est pas terminé. La règle est simple et non négociable.

Anti-pattern 4 — La définition en silo

Le PO écrit les spécifications seul, puis les partage à l'équipe « pour lecture ». L'équipe lit, hoche la tête, et commence à implémenter. Les divergences d'interprétation émergent pendant le sprint — trop tard pour être corrigées sans coût.

Remède : La spécification est un artefact collectif. Elle se rédige ensemble, ou elle est challengée ensemble avant d'être validée. Il n'y a pas de spécification en silo dans le SDLC réconcilié.

> ⚠ **Signal de vigilance — Anti-patterns combinés**
>
> Les anti-patterns SENSE se présentent rarement seuls. L'élicitation superficielle couplée au rush vers FORGE et à la définition en silo est la combinaison la plus fréquente — et la plus destructrice. Elle produit des équipes qui travaillent dur, livrent régulièrement, et s'étonnent de ne jamais atteindre la confiance dans ce qu'elles construisent. Ce n'est pas un problème de vélocité. C'est un problème de fondation.

Transition SENSE → FORGE : critères de passage et signal

La sortie de la zone SENSE et l'entrée dans la zone FORGE est une transition qui doit être consciente, délibérée, et validée. Ce n'est pas un glissement naturel — c'est un moment de décision.

Les critères de passage de SENSE à FORGE sont satisfaits lorsque :

4. La DoR est validée : le besoin est suffisamment compris pour avoir été défini.
5. La DoD-spec est validée : les spécifications produites satisfont les 5 critères (forme SI/ALORS, atomicité, testabilité, lecture partagée, traçabilité).
6. L'équipe peut répondre à la question : « Si nous construisons exactement ce que ces spécifications décrivent, aurons-nous créé de la valeur pour l'utilisateur ? » La réponse doit être oui — avec confiance, pas avec réserves.

Quand ces critères sont satisfaits, le signal de passage est clair. L'équipe entre en FORGE avec quelque chose de précieux : non pas une liste de tâches, mais une compréhension partagée de ce qui mérite d'être construit, formulée de façon à ce que sa construction puisse être vérifiée. C'est la condition de la confiance dans le cycle.

🔍 **Symptômes à observer — Transition SENSE → FORGE prématurée**

- Les développeurs commencent à implémenter avant que les critères d'acceptation soient écrits
- Le QA ne participe pas à la phase SENSE et découvre les spécifications en début de sprint
- La DoR n'a jamais été discutée explicitement lors de l'atelier
- Les spécifications sont rédigées en une phrase sans condition ni résultat observable
- L'équipe « sait à peu près » ce qu'elle va construire mais ne peut pas le formuler précisément

❓ **Questions de réflexion**

- Quel est le dernier atelier SENSE que vous avez conduit ? Quels en étaient les outputs concrets ?
- Votre équipe a-t-elle une DoR explicite et connue de tous les membres ?
- Rania, Théo et Karim étaient-ils tous les trois présents lors de votre dernière session de définition ?
- Avez-vous déjà dit stop à une transition vers FORGE parce que la DoR ou la DoD-spec n'était pas satisfaite ?
- Si votre équipe devait écrire sa DoR aujourd'hui, de zéro, quelles seraient vos 5 conditions ?

💡 **À retenir — SENSE en pratique**

L'atelier SENSE est le moment où la compréhension devient décision, et où la décision devient spécification. Il se déroule en 3 temps (comprendre, nommer, spécifier), mobilise 6 outils, réunit 5 rôles, et se garde de 4 anti-patterns majeurs. Sa sortie est formalisée par deux critères : DoR et DoD-spec. Quand ces deux critères sont satisfaits, l'équipe est prête pour FORGE.

Synthèse — SENSE : ce que cette zone produit

La zone SENSE est terminée. Les deux phases qui la composent — ÉLICITER et DÉFINIR — ont été explorées en profondeur : leurs définitions, leurs limites, leurs techniques, leurs critères de sortie, leurs anti-patterns, et leurs connexions mutuelles.

Voici la synthèse de ce que SENSE produit et de ce qu'elle garantit :

Dimension	ÉLICITER	DÉFINIR
Question principale	Qu'est-ce qui se passe réellement ? Quelle est la vraie douleur ?	Qu'est-ce que le système doit faire de façon observable et testable ?
Output principal	Besoin compris, nommé, validé par tous	Spécifications SI/ALORS, atomiques, testables, tracées
Critère de sortie	DoR — 5 conditions validées	DoD-spec — 5 critères satisfaits
Risque principal si mal faite	Construction de la mauvaise chose	Construction de quelque chose qu'on ne peut pas vérifier
Acteur clé	Facilitateur + représentant métier + PO	PO + QA + Tech Lead
Outils principaux	Entretien structuré, observation, carte d'empathie, atelier de signal	Canvas SI/ALORS, DoD-spec Checklist, validation croisée
Domaine	SPECIFY	SPECIFY

SENSE prépare FORGE non seulement en fournissant des spécifications exploitables — elle prépare aussi la confiance dans les décisions qui vont suivre. Une équipe qui entre en FORGE depuis une SENSE solide sait pourquoi elle construit ce qu'elle construit, comment elle saura que c'est terminé, et comment elle confirmera que la valeur a été créée. C'est une équipe en maîtrise — pas en exécution aveugle.

Récapitulatif — Critères de sortie de la zone SENSE

DoR — Definition of Ready (sortie d'ÉLICITER)

#	Condition	Validée ?
1	La douleur est nommée en une phrase sans solution	☐
2	La valeur est articulée (pour qui, quoi change)	☐
3	Le risque principal est identifié (au moins une hypothèse critique)	☐
4	La décision est claire (qui valide, sur quels critères)	☐
5	Un signal de confirmation est esquissé	☐

DoD-spec — Definition of Done Spécification (sortie de DÉFINIR)

#	Critère	Validé ?
1	Forme SI/ALORS respectée pour chaque spécification	☐
2	Atomicité vérifiée (une condition, un résultat par spécification)	☐
3	Testabilité confirmée par le QA	☐
4	Lecture partagée validée (PO, tech lead, QA lisent la même chose)	☐
5	Traçabilité au besoin établie	☐

Glossaire de la zone SENSE

Terme	Définition dans le SDLC réconcilié
ÉLICITER	Phase 1 du SDLC réconcilié. Recueillir les signaux et comprendre les besoins à travers les 5 objets (douleur, valeur, risque, décision, signal) et les 5 profils de sources.
DÉFINIR	Phase 2 du SDLC réconcilié. Transformer le besoin compris en spécifications observables et testables, formulées en structure SI/ALORS.
SENSE	Zone regroupant ÉLICITER et DÉFINIR. Fait sens du besoin avant d'entrer dans la construction (FORGE).
DoR	Definition of Ready — critère de sortie d'ÉLICITER. 5 conditions qui attestent qu'un besoin est prêt à être défini.
DoD-spec	Definition of Done Spécification — critère de sortie de DÉFINIR. 5 critères qui attestent que les spécifications sont prêtes à entrer en FORGE.
SI/ALORS	Forme canonique d'une spécification ToTAL. SI [condition observable] ALORS [résultat observable attendu].
Spécification atomique	Spécification couvrant exactement une condition, un résultat, un niveau d'observation, et une preuve possible.
Spécification décorative	Énoncé qui ressemble à une spécification mais ne peut pas être testé tel quel. Contient généralement des termes subjectifs non mesurables.
Atelier SENSE	Session de travail collectif en 3 temps (comprendre, nommer, spécifier) qui couvre les phases ÉLICITER et DÉFINIR.
Grande boucle	Connexion de CONFIRMER vers ÉLICITER. Mécanisme d'apprentissage continu qui

Terme	Définition dans le SDLC réconcilié
	enrichit chaque nouveau cycle. Ruban de Möbius : toujours en avant.
Traçabilité ToTAL	Capacité à relier chaque spécification à son signal de besoin source et à son test de vérification correspondant.

→ Vers la Partie B — FORGE : Planifier et Implémenter

La zone SENSE est complète. Vous avez maintenant entre les mains des spécifications solides : observables, testables, tracées. L'équipe DREAM-TEAM — Rania, Théo, Karim — est alignée sur ce qui doit être construit et pourquoi. La DoR et la DoD-spec sont satisfaites.

La Partie B entre dans la zone **FORGE** : le territoire de PLANIFIER et d'IMPLÉMENTER. C'est là que la pensée devient code. Que la spécification devient comportement réel. Que l'architecture rencontre la réalité de l'exécution.

Mais FORGE n'est pas une zone de pure exécution. C'est une zone de décision continue — sur les priorités, sur les risques techniques, sur les compromis architecturaux, sur les incréments à livrer. Une équipe qui entre en FORGE sans maîtrise SENSE exécute dans le brouillard. Une équipe qui entre en FORGE avec une SENSE solide construit avec clarté.

La Partie B vous attend. **FORGE commence ici.**

« SENSE ne prépare pas seulement ce qu'on va construire — il prépare la confiance dans ce qu'on va décider. »

— Collection ToTAL — Réconciliation, Livre 2 sur 21

Quatrième de couverture

Ce fascicule — Partie A de trois — plonge dans la zone SENSE du SDLC réconcilié : les deux premières phases du cycle, ÉLICITER et DÉFINIR. Il expose pourquoi l'élicitation est la phase la plus sous-estimée et la plus critique du développement logiciel, comment les 5 objets d'élicitation et les 5 profils de sources structurent une exploration rigoureuse du besoin, et comment la forme canonique SI/ALORS transforme un besoin compris en spécification observable, testable, et traçable. À travers des scènes narratives, des outils pratiques, des critères de sortie formalisés (DoR, DoD-spec), et les anti-patterns les plus fréquents, cette Partie A donne à toute équipe les fondations pour entrer en FORGE avec confiance et maîtrise.

Auteur : Léo Michel Bélanger, fondateur d'ORIGINALIUM

Collection : ToTAL — Réconciliation · Livre 2 sur 21 · Partie A sur 3

Site : www.beastest.com

Note : BEASTEST est un outil optionnel qui peut faciliter et accélérer les pratiques décrites dans cette collection. Toutes les méthodes présentées sont applicables indépendamment de tout outil ou plateforme.